28/02/26

24BCE5404 ①

Project title:

Design and Implementation of an inventory and Sales Management System for a Retail Store

Normalization:

Normalization is a systematic process in database design used to organize data into well-structured relations (tables) in order to minimize redundancy and eliminate undesirable characteristics such as insertion, update and delete anomalies. It involves decomposing large tables into smaller, related tables while preserving data integrity through proper relationships.

Key objectives

i) Eliminate data redundancy by avoiding unnecessary duplication of data
ii) Ensure data consistency across the database
iii) prevent anomalies during insert, update, and delete operations.

Normal forms:

A Normal Form is a level of database normalization that specifies the constraints a relation must satisfy in order to reduce redundancy, eliminate anomalies, and maintain consistent data dependencies. A table progresses through different normal forms such as 1NF, 2NF, 3NF, BCNF, 4NF, 5NF with each level imposing stricter rules

Types of Normal forms:-

1] First Normal Form (1NF) : Each attribute contains atomic (indivisible) values
* No repeating groups or multivalued attributes

2] second Normal Form (2NF): Table must be in 1NF
* All non-key attributes fully depend on the entire primary key or candidate key

3] third Normal Form (3NF) : Table must be in 2NF
* No transitive dependencies between non-key attributes

4] Boyce - Codd Normal Form (BCNF): stronger version of 3NF
* Every determinant must be a candidate key

Lossless Join (Nonadditive Join property):

The Lossless join property ensures that when a relation is decomposed into two or more smaller relations, the original relation can be perfectly reconstructed by performing a natural join on the decomposed relations without any loss or addition of data

Dependency preservation property:

This property ensures that functional dependencies of the original relation can be enfored by applying constraints on the decomposed relations without needing to join the tables.

Normalization

1] Category:

category_id	Category_name	Description
PK		

Functional dependency:

category_id → category_name, Description

→ category_name depends on category_id

→ Description depends on category_id

candidate key: {category_id}

Prime Attributes: {category_id}

Non prime Attributes: {category_name, description}

1NF (First Normal form)

i) satisfies 1NF because all attributes are atomic (no multivalued or repeating attributes)

ii) Each row has a unique primary key (category_id)

2NF (second Normal form)

i) satisfies 2NF because primary key is single attribute (category_ID) or there is only one candidate key

ii) No partial Dependency exists

3NF (Third Normal Form)

i) No transitive dependency exists

ii) Non key attributes depend only on the primary key

BCNF

i) category-ID is the only determinant

ii) category-ID is a candidate key

∴ The category relation is in BCNF (Boyce-Codd Normal Form) because all non-key attributes (category-Name and Description) are fully functionally dependent on the primary key (category-ID) and there are no partial or transitive dependencies

2] Product:

Product_id (primary key)

Product_name

Unit_price

Barcode

category-id (Foreign key)

Supplier_id (foreign key)

Extra attributes added

HSN-Code (primary key)

GST_Rate

Functional dependencies

product_ID → product_Name, unit-price, Barcode, category_id, supplier_id, HSN-code

Barcode → product_id, product-Name

{HSN-code} → GST_Rate

Candidate keys: Product_ID, Barcode
Prime attributes: {product_ID, Barcode}
Non prime attributes: {product_Name, unit-price, category-id, Supplier-id, HSN-code, GST-Rate};

1NF (first Normal form)

i) satisfies 1NF because all attributes are atomic
ii) No repeating groups

2NF (second Normal form)

i) FD1: product_ID → product_Name, unit-price, Barcode, category-ID, Supplier_id, HSN_code
→ All attributes depend on product_id
→ since the primary key is not composite, partial dependency does not exist
→ therefore, the relation satisfies 2NF
ii) FD2: Barcode → product_id, product_Name
→ A barcode is unique for each product, so no two products share the same barcode
→ This means barcode acts like a candidate key
iii) FD3: {HSN_code → GST-Rate}
→ There is no partial dependency since HSN_Code is not part of candidate key.

NOTE: A partial dependency occurs when a non-prime attribute depends on only part of composite candidate key

3NF (Third Normal form)

i) FD1: product_id → product_name, unit_price, barcode category_id, supplier_id, HSNcode

→ since product_id is a candidate key it is in 3NF

→ Every candidate key is a superkey. Therefore LHS has per key and transitive dependency is not there

ii) FD2: barcode → product_id, product_name

→ barcode is a candidate key, therefore there is no transitive dependency

iii) FD3: {HSN_code} → GST → Rate

LHS is not a superkey and RHS is not a prime attribute

therefore there exists transitive dependency

product_id	product name	unit price	barcode	category id	Supplier id	HSN code	GST Rate
pk				fk	fk		

⇓

Product id	product name	unit price	barcode	category id	supplier id	HSN code
pk				fk	fk	fk

HSN_code	GST_Rate
pk	

BCNF

i) FD1: product_id → product_Name, unit_price, Barcode, category_id, supplier_id

→ product_id is a superkey so it is in BCNF

ii) FD2: Barcode → product_id, product_Name

→ Barcode is a superkey so it is in BCNF

iii) FD3: {HSN_code → GST_Rate}

→ HSN_code is a superkey in a new relation so it is in BCNF. Therefore all the functional dependencies are in BCNF

9] Supplier

Attributes:

→ supplier_id (primary key)
→ supplier_name
→ phone (Multivalued)
→ address (composite)
→ Email

Extra attributes added:

→ Bank_account_Number
→ IFSC_code
→ Branch_Name
→ Bank_Name

Supplier bank account details, so that who buys the products can able to do payment through these details.

Functional dependencies:-

i) Supplier_id → phone_Number

ii) Supplier_id → Supplier_name, address, email, Bank_account_Number

iii) Bank_account_Number → IFSC code, Branch_name, Bank_name

iv) pincode → , city, state

Decompose into 1NF

Supplier ID	supplier name	city	state	pincode	Bank Account Number	branch name	Bank name
PK							

Supplier_Id	Phone_Number
PK	PK

→ Satisfies 1NF because all attributes are atomic

→ Multivalued attributes such as (phone_Number) is joined with supplier_Id (primary key) attribute and created a seperate relation to avoid redundancy of data

→ Composite attributes such as address (street, city, state, pincode) are made into seperate columns.

Candidate keys: {supplier_Id}

Prime attributes: {supplier_ID}

Non-prime attributes: {supplier_name, street, city, state, pincode, Account_Number, IFSC_code, Branch_name, Bank_name}

24BCE5404 ⑤

2NF (Second Normal form)

→ There is no partial dependency since there is only one candidate key all the functional dependencies are in 2NF.

3NF (third Normal form)

→ Supplier_id → supplier_name, pincode, email, bank_account_number

pincode → city, state

→ There is a transitive dependency since pincode is not a superkey and street, city, state are not prime attributes so it violates 3NF. So we have to decompose the relation

Supplier Id	Supplier name	city	state	pincode	Bank Account Number	IFSC code	Branch name	Bank name
PK								

⇓

Supplier_id	Supplier name	pincode	Bank Account Number	IFSC code	Branch name	Bank name
PK		FK				

pincode	city	state
PK		

→ Supplier_id → Supplier_name, pincode, email, Bank_account_number

Bank_account_number → {IFSCcode, Branch_name, Bank_name}

NOTE: SBIN0012345 → IFSC code

SBIN → Bankname

012345 → Branchcode to determine specific branch of bank

→ There is a transitive dependency because account_number is not a superkey and IFSCcode, Branch_name, Bank_name are not prime attributes. So it violates 3NF

Decomposing the relation to make into 3NF

Supplier_id	supplier name	pincode	Bank Account Number	IFSC code	Branch name	Bank name
pk		fk				

Supplier_id	Supplier_name	pincode	Bank_Account_Number
PK		fk	fk

Bank_Account Number	IFSC code	Branch_name	Bank_name
PK			

24BCE5404 (6)

BCNF

Functional dependencies after decomposition:

FD1: Supplier_id ⟶ Supplier_name, pincode, email, Bank_account_Number

FD2: Supplier_id ⟶ phone_Number

FD3: Bank_account_Number ⟶ IFSC_code, branch_name, Bank_name

FD4: pincode ⟶ , city, state

FD1 is in BCNF because supplier_id is a superkey; it has superkey in RHS of functional dependency

Hence it is in BCNF

FD2 is in BCNF because supplier_id is a superkey

FD3 is in BCNF Bank_account_Number is a superkey in a decomposed Relation

FD4 is in BCNF because pincode is a superkey in a decomposed Relation

4) Inventory

Inventory_ID (PK)

product_ID (FK)

stock_in

stock_out

current_stock

Reorder_level

NOTE: A ware house is a storage location, It is where your company keeps products after receiving from suppliers and before selling or distributing them

Extra attributes added

Expiry_date

warehouse_ID

warehouse_Manager

warehouse_address (city, state, pincode)

product_description

Functional dependencies:

(Inventory_ID, product_id) → stock_in, stock_out, expiry_date, current_stock, reorder_level, warehouse_id

product_Id → product_description

(Warehouse_id) → warehouse_Location, warehouse_address, warehouse_manager

pincode → , city, state.

(product_ID, warehouse_ID) → Reorder_level

candidate key: {Inventory_ID, product_ID}

Explanation:

product_id	stock_in	Expiry_Date
P001	100	2026-03-01
P001	50	2026-04-01

Here, the same product appears in multiple rows with different stock/expiry

If product_id alone were the pk, the database cannot distinguish these two rows, causing data loss or conflicts.

→ (Inventory_ID, Product_ID)

Inventory_ID = unique identifier for each batch/entry

Product_ID = identifies the product itself

Together (Inventory_ID, Product_ID) uniquely identifies each inventory record, even if the same product appears multiple times with different expiry dates

Inventory_Id	product_id	Expiry_date	stock_in
INV001	P001	2026-03-01	100
INV002	P001	2026-04-01	50

candidatekey = (Inventory_ID, product_ID)

Each row is now unique,

1NF

I_ID	P_ID	stock in	Stock out	Current	Reorder	Exp date	W_ID	W_Manager	city	State	pin code	desc
pk	pk, fk											

→ all atributes are atomic

→ Composite attributes such as address (street, city, state) pincode) are made into seperate columns

2NF (Second Normal form)

FD1: (Inventory_ID, product_Id) → stock_in, stock_out, expiry_date, current_stock, reorder_level, warehouse_id

already in 2NF because LHS is candidate key. And every non-prime attributes in RHS is functionally dependent on LHS.

FD2: product_ID → product_description

product_ID: part of candidate key

product_description: non-prime attributes

Non-prime attributes are partially dependent on the primary key so it violates 2NF. we have to decompose into 2NF

I_ID	P_ID	stock in	stock out	current	Reord	EXP date	W_ID	W_Manager	city	state	pin code	des
pk	pk,fk											

⇓

I_ID	P_ID	Stock in	Stock out	current	Reorder	Expdate	W ID	W-Manager	city	state	pin Code
PK	PK,fk										

product_ID	description
PK,fk	

24BCE5404 (8)

3NF (3rd Normal form)

FD1: Does not violate 3NF because {Inventory_ID, product_Id} is a candidate key

FD2: Does not violate 3NF

FD3: {warehouse_id} → warehouse_location, street, city, state, pincode, warehouse_Manager

warehouse_id is not a Superkey and warehouse_Manager, warehouse_location are not prime attributes so it violates 3NF. And there exists a transitive dependency

I_ID	P_Id	stock in	stock out	current	Reorder	Exp date	W_ID	W_Manager	city	state	pincode
PK	PK						PK, FK				

⇓

Inventory_ID	product_Id	stock in	stock out	current stock	Reorder	Exp date	warehouse ID
PK	PK, FK						FK

warehouse ID	warehouse Manager	street	city	state	pincode
PK					

FD4: pincode → city, state

pincode is not superkey and street, city, state are not prime attributes. There exists a transitive depency so it violates 3NF

warehouse_id	warehouse_Manager	city	state	pincode
PK				

⇓

warehouse_id	warehouse_Manager	pincode
PK		FK

pincode	city	state
PK		

FDs: (productid, warehouse_id) → reorder_level

Note: Note a 2NF violation (partial dependency applies only to subsets of the candidatekey)

It violates 3NF as (product_ID, warehoube_ID) is not a Super key in Inventory and reorder_level is not a prime attribute so it has transitive dependency violates 3NF

Inventory_Id	product_Id	Stock in	stock out	current stock	reorder	Exp_date	warehouse ID
PK	FK						FK

⇓

Inventory_Id	product_Id	stock in	Stock out	Current stock	Expiry date	warehouse Id
PK	FK					FK

product_Id	warehouse_Id	Reorderlevel
PK, FK	PK, FK	

24BCE5404 (9)

5) Purchase_order

Attributes:

1] po_id
2] supplier_id
3] order_date
4] total_amount
5] status

Extra attributes

1] Expected_delivery_date
2] payment_status

Functional dependencies

PO_ID → supplier_id, order_date, total_amount, status, delivery_date, payment_status

status → payment_status

candidate key : {PO_ID}

1NF (1st normal form)

FD1: PO_ID → supplier_id, order_date, total_amount, status, delivery_date, payment_status

FD2: status → payment_status

* all attributes are atomic no multivalued attributes so it is in 1NF

2NF (2nd Normal form)

FD1: Since PO_ID is the candidate there is no partial dependency

FD2: status is not a part of candidate key

3NF (3rd Normal form)

FD1: po_ID is a candidate key so there is no transitive dependency

FD2: status → payment_status

po_ID → status

∴ po_ID → payment_status

there exist a transitive dependency so it violates 3NF

status	payment_status
ordered	pending
shipped	pending
received	paid

Here payment_status is completely dependent on status of the order

po_Id	Supplier_id	order_date	total_amount	status	delivery date	payment status
PK	fk					

⇓

po_Id	Supplier_id	order_date	total_amount	status	delivery date
PK	fk			fk	

status	payment_status
PK	

BCNF

Both functional dependencies are in BCNF

24BCE5404 (10)

6) Purchase_order_details

po_id (foreign key)
product_id (foreign key)
quantity_ordered
unit_price
line_total
batch_no.

Added attributes

purchase_agent_name – Name of the agent who placed order
shipping_mode – Air/sea/Road

Functional dependencies:

1) {po_id, product_id} → {quantity_ordered, unit_price, batch_no}

2) {po_id} → {purchase_agent_name, shipping_mode}

3) {quantity, unit_price} → {line_total} ↓
derived dependency violates 3NF

Candidate key = {po_id, product_id} (Both combinedly is a candidate key)
prime attributes = {po_id, product_id}

1NF

1) All the functional dependencies satisfies 1NF as there is no multivaled attributes and all attributes are atomic

2NF

FD2: {po_id} → {purchase_agent_name, shipping_mode}

non key attributes are dependent on only part of primary key so there exists a partial dependency so it violates 2NF

decompose the table:

po_id	product_id	quantity ordered	unit_price	line_total	batch no	purchase agent name	shipping mode
pk, fk	pk, fk						

⇓

po_id	product_id	quantity ordered	unit_price	line_total	batchno
pk, fk	pk, fk				

po_id	purchase_agent_name	shipping_mode
pk, fk		

3NF

FD3: {quantity, unit_price} → {line_total}

The column line_total (total price) is a derived attribute because its value is calculated from other non-key columns (quantity_order × unit_price). storing it in database causes data redundancy and can lead to inconsistencies if the other columns are updated but line_total is not. So remove the line_total column from the physical database storage. there is no need to store a value that can be derived from existing data.

NOTE

24UBCE5404 (11)

po_id	product_id	quantity_ordered	unit_price	batchno
pk,fk				

NOTE: line_total is for that particular product where as total_amount in purchase_order is for total_amount in one purchase_order

BCNF

1] Both functional dependencies are in BCNF since both LHS is primary keys.

7] Employee:

EmployeeID
EmployeeName
Role
phoneNumber (not multivalued attribute)
address → composite

Functional dependencies:

EmployeeID → EmployeeName, role, phoneNumber, address

candidate key: {EmployeeID}

1NF (first Normal form)

→ If any column contains composite attribute decompose it into atomic attributes

Employee_ID	Employee name	Role	phone number	city	State	pincode
PK						

2NF (Second Normal form)

→ There is only one primary key no partial dependency

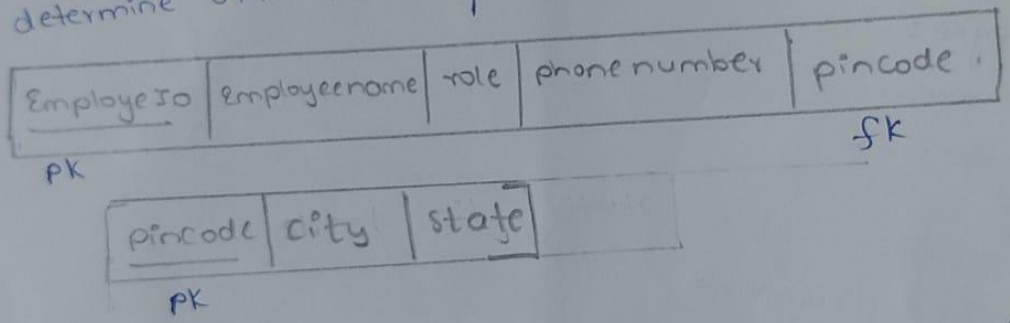
3NF (Third Normal form)

→ Employee_ID → {employeeName, role, phonenumber, street, city, state, pincode}

pincode → {city, state}

Street, city, state are not directly dependent on the primary key (Employee_ID) but on another non-key attribute (PINCODE)

Violation of 3NF : IN 3NF, non-key attributes cannot determine other non-key attributes



BCNF

→ FD1 : satisfies BCNF because employee_ID is a superkey

FD2 : statisfies BCNF because pincode is a superkey in another relation.

24BCE5404 (12) (12)

8) Customer

customer_id

customer_name

phone (not multivalued attribute)

email

Functional dependencies:

customer_id ⟶ {customer_name, phone, email}

Candidate key: {customer_id}

⇒ there is no multivalued attributes, no partial dependency, no transitive dependency and customer_id is a superkey hence it is in BCNF

9) Sales_transaction:

i) transaction_id (primary key)

ii) customer_id

iii) employee_id

iv) transaction_date

v) grand_total - total amount of whole transaction

Extra attributes added

i) payment_type_id: A numeric code that stands for a specific way the customer paid

Ex: 1) (stands for cash)

2) (stands for creditCard)

3) (stands for UPI/ online)

ii) payment_processing_fee

functional dependencies:

transaction_id ⟶ customer_id, employee_id, transaction_date, grand_total, payment_id, type_id, payment_processing_fee

payment_type_id ⟶ payment_processing_fee

candidate key: {transaction_id}

1NF (first Norm)

→ Two relations are in 1NF as there only atomic values in each column and there are no multivalued attributes

2NF (second Normal form)

Since the primary key is single-attribute (transaction_id), no partial dependency exist

3NF (Third Normal form)

payment_type_id ⟶ payment_processing_fee

* payment_processing_fee depends on payment_type_id, not directly on transaction_id. This is a transitive dependency

transaction id	customer id	Employee_id	transaction date	grand total	payment type_id	payment processing
PK	FK	FK				

⇓

transaction id	customer id	Employee id	transaction date	grand total	payment type_id
PK	FK	FK			FK

payment_type_id	payment_processing_fee

pk

BCNF

→ FD1: transaction_id is a primary key in relation1 so it is in BCNF

→ FD2: payment_type_id is a primary key in relation2 so it is in BCNF

10) Sales_Transaction_Details:

transaction_id (foreign key)

product_id (foreign key)

quantity_sold

unit_price

line_total (derived attribute)

Extra_attributes added

coupon_code

tax_category

tax_rate

Functional dependencies:

1) (transaction_id, product_id) → quantity_sold, unit_price, line_total, tax_category, tax_rate

2) transaction_id → coupon_code

3) (quantity_sold, unit_price) → line_total

4) tax_category → tax_rate

candidate key : (transaction_id, product_id)

1NF (first Normal form)

→ All the columns in a relation are atomic no multivalued or composite attributes

2NF (second Normal form)

→ tranction_id → coupon_code

coupon_code non-prime attribute dependent on only one part of candidate key (primary key) so there exist a partial dependency and violates 2NF

Decompose the table to have transaction_id as primary-key.

transaction id	product id	quantity sold	unit price	line total	tax category	tax rate
PK, FK						

transaction_id	coupon_code
PK	

coupon_code is applied once per transaction, not per product. this means all rows for the same transaction_id will have the same coupon_code, regardless of product_id. so coupon_code depends on transaction_id only, not on the fully composite key (transaction_id + product_id)

tax_category	tax_rate

PK

BCNF

FD1: {transaction_id, product_id} is a superkey hence it is in BCNF

FD2: transaction_id is a superkey in relation2 so it in BCNF

FD3: tax_category is also superkey in relation3 so it is in BCNF

ii) payment (sales)

i) payment_id
ii) transaction_id
iii) payment_method
iv) amount_paid
v) payment_date

Functional dependencies

payment_id ⟶ {transaction_id, payment_method, amount_paid, payment_date}

candidate_key: payment_id

since there is only one attribute in candidate key it satisfies 1NF (no multivalued attributes and composite attribute broken down into respespective subcolumns) and satisfies 2NF (no partial dependency) and also satisfies 3NF (no transitive dependency)

The relation is in BCNF because payment_id is a super key.

24BCE5404 (14)

3NF (third Normal form)

i) (quantity-sold, unit-price) → line_total

line_total is a derived_attribute (line_total = quantity_sold x unit_price). Since its value can always be computed from existing data, storing it is redundant. line_total should not be stored in the Physical table. It should be calculated during queries whenever needed. This keeps the database consistent, normalized and efficient

transaction id	product id	quantity sold	unit price	line total	tax_category	tax rate
pk, fk	fk, pk					

⇓

transaction id	product id	quantity sold	unit price	tax category	tax rate
pk	fk, pk				

ii) tax_category → tax_rate

tax_category is not a superkey and tax_rate is not a prime attribute so there exist a transitive. Decompose the table to have tax_category as primary key or superkey.

transaction id	product id	quantity sold	unit price	tax category	tax rate
pk, fk	fk, pk				

⇓

transaction id	product id	quantity sold	unit price	tax_category
pk, fk	pk, fk			fk

24BCE5404 (15)

12) stock_adjustment

i) Adjustment ID (primary key)
ii) Product ID
iii) Employee ID
iv) Adjustment Type
v) Quantity Adjusted
vi) Reason

Functional dependencies:

i) Adjustment ID → Product ID, Employee ID, Adjustment type, Quantity Adjusted, Reason

Candidatekey: {Adjustment ID}

This functional dependency satisfies 1NF as there are no composite and Multivalued attributes.
It satisfies 2NF, because there is no partial dependency as there is only one attribute in primary key
It satisfies 3NF because there is no transitive dependency
Hence it is in BCNF having adjustment ID as superkey.

Verification: Check for redundant and extraneous attributes and check whether all entities are in BCNF

1) Category → satisfies BCNF
2) Product → satisfies BCNF
3) HSN_Tax → satisfies BCNF
4) Supplier → satisfies BCNF
5) Supplier_bank_details → satisfies BCNF
6) Supplier_contact → satisfies BCNF
7) Supplier_address → satisfies BCNF
8) Inventory → BCNF. Assuming current stock attribute does not depend on stock_in and stock_out. It may change independently (manual adjustments, corrections, damages, etc), so it cannot be computed just from stock_in and stock_out reliably
9) Product_reorder → satisfies BCNF
10) warehouse → satisfies BCNF
11) warehouse_address → satisfies BCNF
12) product_description → satisfies BCNF
13) purchase_order → total_amount does not dependent on quantity x unit_price. GST and extra charges are added to total_amount
14) purchase_order_status → satisfies BCNF
15) purchase_order_details → In this table it contains unit_price that can be calculated from product table

So it is removed here

po_id	product_id	quantity_ordered	unit_price	batch_NO
PK, fK	PK, fK			

⇓

po_id	product_id	quantity_ordered	batch_NO
PK, fK	PK, fK		

15) Delivery_details :→ satisfies BCNF

17) Employee_address → satisfies BCNF

18) Customer → satisfies BCNF

19) Sales_transaction → grand total does not depend on (quantity sold x unit price) so it is dep not dependent attribute

20) Employee → satisfies BCNF

21) Sales_transaction_fees → satisfies BCNF

22) Sales_transaction_details → Here unit_price is unit selling price ≠ unit_price (in product table)

23) Sales_transaction_taxRates → satisfies BCNF

24) payment → satisfies BCNF

25) stockadjustment → satisfies BCNF

(17)

Normalized tables

1] category:

category_id	category_name	description
pk		

2) product

product id	product name	barcode	unit price	category id	supplier id	HSN code
pk				fk	fk	fk

3) HSN_TAX

HSN_code	GST_Rate
pk	pk fk

4) Supplier

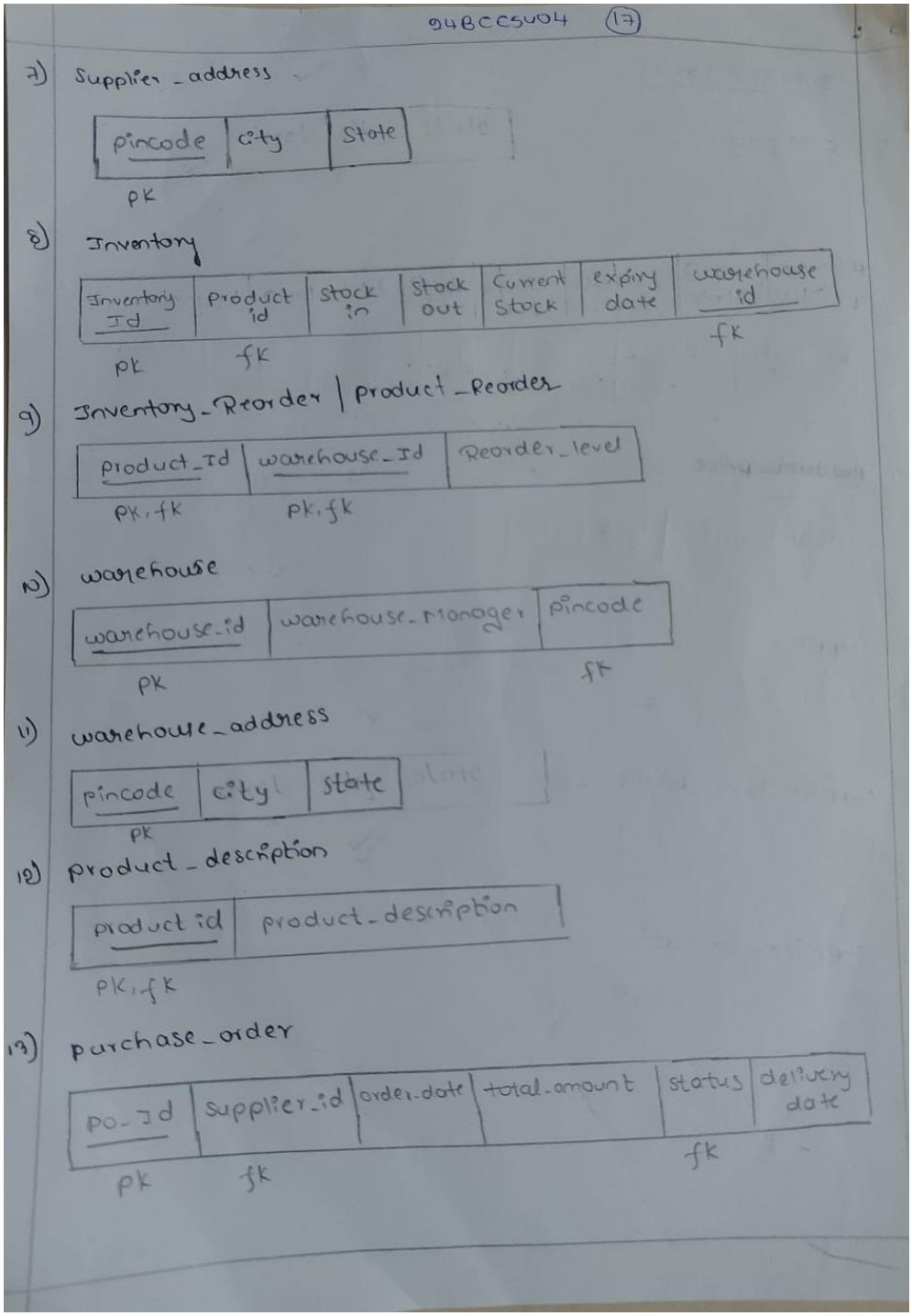
supplier_id	supplier_name	pincode	Bank_ACcount number
pk		fk	fk

5) supplier_bank_details

Bank_Account_number	IFSC code	Branch_name	Bank_name
pk			

6) supplier_contact

Supplier_id	phone
pk	pk


24BCE5404 (17)
7) Supplier_address
pincode | city | State
PK
8) Inventory
Inventory Id | Product id | stock in | stock out | Current Stock | expiry date | warehouse id
PK
FK
FK
9) Inventory_Reorder | Product_Reorder
Product_Id | warehouse_Id | Reorder_level
PK, FK
PK, FK
10) warehouse
warehouse_id | warehouse_Manager | pincode
PK
FK
11) warehouse_address
pincode | city | state
PK
12) Product_description
product id | product_description
PK, FK
13) purchase_order
PO_Id | supplier_id | order_date | total_amount | status | delivery date
PK
FK
FK

14) purchase_order-status

status	payment_status
PK	

15) purchase_order - details

po_id	product_id	quantity_ordered	batch_no
PK, fk	PK, fk		

16) Delivery-details

po_id	purchase_agent_name	shipping_mode
pk, fk		

17) Employee

Employee_Id	Employee name	Role	phone_number	pincode
pk				fk

18) Employee_address

pincode	city	state
pk		

19) customer

customer_id	customer_name	phone	email
pk			

20) Sales_transaction

transaction_id	customer id	employee id	transaction date	grand total	payment type_id
pk	fk	fk			fk

21) Sales_transaction_fees

payment_type_id	payment_processing_fee
PK	

22) Sales_transaction_details

transaction id	product id	quantity sold	unit price	tax_category
PK, fk	PK, fk			fk

23) Sale_transaction_taxRates

tax_category	tax_rate
PK	

24) payment

payment_id	Tranction_id	payment_Method	Amount_paid	payment_dat
PK	fk			

25) stock_adjustment

Adjustment_id	Product id	Employee id	Adjustment type	Quantity adjusted
PK	fk	fk		

DATABASE IMPLEMENTATION:

```
SQL> CREATE TABLE Category(
  2      Category_id NUMBER PRIMARY KEY,
  3      Category_name VARCHAR2(50),
  4      Description VARCHAR2(200)
  5  );

Table created.

SQL> INSERT INTO Category VALUES(1,'Groceries','Daily food items');

1 row created.

SQL> INSERT INTO Category VALUES(2,'Beverages','Drinks and juices');

1 row created.

SQL> INSERT INTO Category VALUES(3,'Personal Care','Hygiene products');

1 row created.

SQL> INSERT INTO Category VALUES(4,'Snacks','Packaged snack items');

1 row created.

SQL> INSERT INTO Category VALUES(5,'Household','Cleaning products');

1 row created.
```

```
SQL> select *from Category;

CATEGORY_ID CATEGORY_NAME                                      DESCRIPTION
----------- -------------------------------------------------- ----------------------------------------------------------------------------------------------------
         1 Groceries                                          Daily food items
         2 Beverages                                          Drinks and juices
         3 Personal Care                                      Hygiene products
         4 Snacks                                             Packaged snack items
         5 Household                                          Cleaning products
```

```
SQL> CREATE TABLE HSN_Tax(
  2      HSN_Code NUMBER PRIMARY KEY,
  3      GST_Rate NUMBER(5,2)
  4  );

Table created.

SQL> INSERT INTO HSN_Tax VALUES(1001,5);

1 row created.

SQL> INSERT INTO HSN_Tax VALUES(2001,12);

1 row created.

SQL> INSERT INTO HSN_Tax VALUES(3001,18);

1 row created.

SQL> INSERT INTO HSN_Tax VALUES(4001,28);

1 row created.

SQL> INSERT INTO HSN_Tax VALUES(5001,0);

1 row created.

SQL> Select *from HSN_Tax;

  HSN_CODE   GST_RATE
---------- ----------
      1001          5
      2001         12
      3001         18
      4001         28
      5001          0
```

```
SQL> CREATE TABLE Supplier_Address(
  2      Pincode NUMBER PRIMARY KEY,
  3      City VARCHAR2(50),
  4      State VARCHAR2(50)
  5  );

Table created.

SQL> INSERT INTO Supplier_Address VALUES(600001,'Chennai','Tamil Nadu');

1 row created.

SQL> INSERT INTO Supplier_Address VALUES(560001,'Bangalore','Karnataka');

1 row created.

SQL> INSERT INTO Supplier_Address VALUES(500001,'Hyderabad','Telangana');

1 row created.

SQL> INSERT INTO Supplier_Address VALUES(400001,'Mumbai','Maharashtra');

1 row created.

SQL> INSERT INTO Supplier_Address VALUES(700001,'Kolkata','West Bengal');

1 row created.

SQL> Select *from Supplier_Address;

   PINCODE CITY                                               STATE
---------- -------------------------------------------------- --------------------------------------------------
    600001 Chennai                                            Tamil Nadu
    560001 Bangalore                                          Karnataka
    500001 Hyderabad                                          Telangana
    400001 Mumbai                                             Maharashtra
    700001 Kolkata                                            West Bengal

SQL>
```

```
SQL> CREATE TABLE Supplier_Bank_Details(
  2      Bank_Account_Number VARCHAR2(20) PRIMARY KEY,
  3      IFSC_Code VARCHAR2(20),
  4      Branch_Name VARCHAR2(50),
  5      Bank_Name VARCHAR2(50)
  6  );

Table created.

SQL> INSERT INTO Supplier_Bank_Details VALUES
  2  ('123456789012','SBIN000111','Chennai Main Branch','SBI');

1 row created.

SQL> INSERT INTO Supplier_Bank_Details VALUES
  2  ('234567890123','ICIC000222','Bangalore MG Road Branch','ICICI');

1 row created.

SQL> INSERT INTO Supplier_Bank_Details VALUES
  2  ('345678901234','HDFC000333','Hyderabad Secunderabad Branch','HDFC');

1 row created.

SQL> INSERT INTO Supplier_Bank_Details VALUES
  2  ('456789012345','AXIS000444','Mumbai Andheri Branch','AXIS');

1 row created.

SQL> INSERT INTO Supplier_Bank_Details VALUES
  2  ('567890123456','KKBK000555','Kolkata Park Street Branch','Kotak');

1 row created.

SQL> select *from Supplier_Bank_Details;

BANK_ACCOUNT_NUMBER  IFSC_CODE            BRANCH_NAME                                        BANK_NAME
-------------------- -------------------- -------------------------------------------------- --------------------------------------------------
123456789012         SBIN000111           Chennai Main Branch                                SBI
234567890123         ICIC000222           Bangalore MG Road Branch                           ICICI
345678901234         HDFC000333           Hyderabad Secunderabad Branch                      HDFC
456789012345         AXIS000444           Mumbai Andheri Branch                              AXIS
567890123456         KKBK000555           Kolkata Park Street Branch                         Kotak
```

```
SQL> CREATE TABLE Supplier(
  2      Supplier_id NUMBER PRIMARY KEY,
  3      Supplier_name VARCHAR2(50),
  4      Pincode NUMBER,
  5      Bank_Account_Number VARCHAR2(20),
  6      FOREIGN KEY(Pincode)
  7      REFERENCES Supplier_Address(Pincode),
  8      FOREIGN KEY(Bank_Account_Number)
  9      REFERENCES Supplier_Bank_Details(Bank_Account_Number)
 10  );

Table created.

SQL> INSERT INTO Supplier VALUES
  2  (1,'Ramesh Kumar',600001,'123456789012');

1 row created.

SQL> INSERT INTO Supplier VALUES
  2  (2,'Suresh Traders',560001,'234567890123');

1 row created.

SQL> INSERT INTO Supplier VALUES
  2  (3,'Anita Enterprises',500001,'345678901234');

1 row created.

SQL> INSERT INTO Supplier VALUES
  2  (4,'Vijay Distributors',400001,'456789012345');

1 row created.

SQL> INSERT INTO Supplier VALUES
  2  (5,'Karthik Wholesale',700001,'567890123456');

1 row created.

SQL> select *from Supplier;

SUPPLIER_ID SUPPLIER_NAME                                      PINCODE BANK_ACCOUNT_NUMBER
----------- -------------------------------------------------- ------- --------------------
          1 Ramesh Kumar                                       600001 123456789012
          2 Suresh Traders                                     560001 234567890123
          3 Anita Enterprises                                  500001 345678901234
          4 Vijay Distributors                                 400001 456789012345
          5 Karthik Wholesale                                  700001 567890123456

SQL>
```

```
SQL> CREATE TABLE Supplier_Contact(
  2       Supplier_id NUMBER,
  3       Phone NUMBER(10),
  4       PRIMARY KEY(Supplier_id,Phone)
  5  );

Table created.

SQL> INSERT INTO Supplier_Contact VALUES(1,9876543210);

1 row created.

SQL> INSERT INTO Supplier_Contact VALUES(1,9123456789);

1 row created.

SQL> INSERT INTO Supplier_Contact VALUES(2,9012345678);

1 row created.

SQL> INSERT INTO Supplier_Contact VALUES(3,9988776655);

1 row created.

SQL> INSERT INTO Supplier_Contact VALUES(4,9090909090);

1 row created.

SQL> Select *from Supplier_Contact;

SUPPLIER_ID      PHONE
----------- ----------
          1 9876543210
          1 9123456789
          2 9012345678
          3 9988776655
          4 9090909090
```

```
SQL> CREATE TABLE Product(
  2      Product_id NUMBER PRIMARY KEY,
  3      Product_name VARCHAR2(50),
  4      Barcode NUMBER UNIQUE,
  5      Unit_price NUMBER,
  6      Category_id NUMBER,
  7      Supplier_id NUMBER,
  8      HSN_Code NUMBER,
  9      FOREIGN KEY(Category_id) REFERENCES Category(Category_id),
 10      FOREIGN KEY(Supplier_id) REFERENCES Supplier(Supplier_id),
 11      FOREIGN KEY(HSN_Code) REFERENCES HSN_Tax(HSN_Code)
 12  );

Table created.

SQL> INSERT INTO Product VALUES (101, 'Rice 5kg', 11111, 350, 1, 1, 1001);

1 row created.

SQL> INSERT INTO Product VALUES (102, 'Apple Juice', 22222, 120, 2, 2, 2001);

1 row created.

SQL> INSERT INTO Product VALUES (103, 'Shampoo', 33333, 180, 3, 3, 3001);

1 row created.

SQL> INSERT INTO Product VALUES (104, 'Biscuits', 44444, 40, 4, 4, 1001);

1 row created.

SQL> INSERT INTO Product VALUES (105, 'Detergent', 55555, 220, 5, 5, 3001);

1 row created.

SQL> SET LINESIZE 200;
SQL> select *from product;

PRODUCT_ID PRODUCT_NAME                                        BARCODE UNIT_PRICE CATEGORY_ID SUPPLIER_ID   HSN_CODE
---------- -------------------------------------------------- ---------- ---------- ----------- ----------- ----------
       101 Rice 5kg                                            11111        350           1           1       1001
       102 Apple Juice                                         22222        120           2           2       2001
       103 Shampoo                                             33333        180           3           3       3001
       104 Biscuits                                            44444         40           4           4       1001
       105 Detergent                                           55555        220           5           5       3001

SQL>
```

```
SQL> CREATE TABLE Product_Description(
  2      Product_id NUMBER PRIMARY KEY,
  3      Product_description VARCHAR2(200),
  4      FOREIGN KEY(Product_id) REFERENCES Product(Product_id)
  5  );

Table created.

SQL> INSERT INTO Product_Description VALUES(101,'Premium rice quality');

1 row created.

SQL> INSERT INTO Product_Description VALUES(102,'1L apple juice bottle');

1 row created.

SQL> INSERT INTO Product_Description VALUES(103,'Anti dandruff shampoo');

1 row created.

SQL> INSERT INTO Product_Description VALUES(104,'Crunchy biscuits pack');

1 row created.

SQL> INSERT INTO Product_Description VALUES(105,'Washing detergent powder');

1 row created.

SQL> select *from Product_Description;

PRODUCT_ID PRODUCT_DESCRIPTION
---------- --------------------------------------------------------------------------------------------------------------------------------------------------------
       101 Premium rice quality
       102 1L apple juice bottle
       103 Anti dandruff shampoo
       104 Crunchy biscuits pack
       105 Washing detergent powder

SQL>
```

```
SQL> CREATE TABLE Warehouse_Address(
  2      Pincode NUMBER PRIMARY KEY,
  3      City VARCHAR2(50),
  4      State VARCHAR2(50)
  5  );

Table created.

SQL> INSERT INTO Warehouse_Address VALUES(600070,'Chennai','Tamil Nadu');

1 row created.

SQL> INSERT INTO Warehouse_Address VALUES(560070,'Bangalore','Karnataka');

1 row created.

SQL> INSERT INTO Warehouse_Address VALUES(500070,'Hyderabad','Telangana');

1 row created.

SQL> INSERT INTO Warehouse_Address VALUES(400070,'Mumbai','Maharashtra');

1 row created.

SQL> INSERT INTO Warehouse_Address VALUES(700070,'Kolkata','West Bengal');

1 row created.

SQL> select *from Warehouse_Address;

   PINCODE CITY                                               STATE
---------- -------------------------------------------------- --------------------------------------------------
    600070 Chennai                                            Tamil Nadu
    560070 Bangalore                                          Karnataka
    500070 Hyderabad                                          Telangana
    400070 Mumbai                                             Maharashtra
    700070 Kolkata                                            West Bengal
```

```
SQL> CREATE TABLE Warehouse(
  2      Warehouse_id NUMBER PRIMARY KEY,
  3      Warehouse_Manager VARCHAR2(50),
  4      Pincode NUMBER,
  5      FOREIGN KEY(Pincode) REFERENCES Warehouse_Address(Pincode)
  6  );

Table created.

SQL> INSERT INTO Warehouse VALUES(1,'Mahesh Varma',600070);

1 row created.

SQL> INSERT INTO Warehouse VALUES(2,'Deepak Sharma',560070);

1 row created.

SQL> INSERT INTO Warehouse VALUES(3,'Naveen Reddy',500070);

1 row created.

SQL> INSERT INTO Warehouse VALUES(4,'Rajesh Patil',400070);

1 row created.

SQL> INSERT INTO Warehouse VALUES(5,'Subhajit Das',700070);

1 row created.

SQL> select *from Warehouse;

WAREHOUSE_ID WAREHOUSE_MANAGER                                  PINCODE
------------ -------------------------------------------------- ----------
           1 Mahesh Varma                                       600070
           2 Deepak Sharma                                      560070
           3 Naveen Reddy                                       500070
           4 Rajesh Patil                                       400070
           5 Subhajit Das                                       700070

SQL>
```

```
SQL> CREATE TABLE Inventory(
  2      Inventory_id NUMBER PRIMARY KEY,
  3      Product_id NUMBER,
  4      Stock_in NUMBER,
  5      Stock_out NUMBER,
  6      Current_stock NUMBER,
  7      Expiry_date DATE,
  8      Warehouse_id NUMBER,
  9      FOREIGN KEY(Product_id) REFERENCES Product(Product_id),
 10      FOREIGN KEY(Warehouse_id) REFERENCES Warehouse(Warehouse_id)
 11  );

Table created.

SQL> INSERT INTO Inventory VALUES
  2  (1,101,200,50,142,TO_DATE('20-12-2026','DD-MM-YYYY'),1);

1 row created.

SQL> INSERT INTO Inventory VALUES
  2  (2,102,100,20,75,TO_DATE('10-10-2026','DD-MM-YYYY'),2);

1 row created.

SQL> INSERT INTO Inventory VALUES
  2  (3,103,150,40,118,TO_DATE('05-11-2026','DD-MM-YYYY'),3);

1 row created.

SQL> INSERT INTO Inventory VALUES
  2  (4,104,300,100,193,TO_DATE('01-09-2026','DD-MM-YYYY'),4);

1 row created.

SQL> INSERT INTO Inventory VALUES
  2  (5,105,250,70,169,TO_DATE('15-08-2026','DD-MM-YYYY'),5);

1 row created.

SQL> select *from Inventory;

INVENTORY_ID PRODUCT_ID   STOCK_IN  STOCK_OUT CURRENT_STOCK EXPIRY_DA WAREHOUSE_ID
------------ ---------- ---------- ---------- ------------- --------- ------------
           1        101        200         50           142 20-DEC-26            1
           2        102        100         20            75 10-OCT-26            2
           3        103        150         40           118 05-NOV-26            3
           4        104        300        100           193 01-SEP-26            4
           5        105        250         70           169 15-AUG-26            5
```

```
SQL> CREATE TABLE Product_Reorder(
  2      Product_id NUMBER,
  3      Warehouse_id NUMBER,
  4      Reorder_Level NUMBER,
  5      PRIMARY KEY(Product_id,Warehouse_id),
  6      FOREIGN KEY(Product_id) REFERENCES Product(Product_id),
  7      FOREIGN KEY(Warehouse_id) REFERENCES Warehouse(Warehouse_id)
  8  );

Table created.

SQL> INSERT INTO Product_Reorder VALUES(101,1,50);

1 row created.

SQL> INSERT INTO Product_Reorder VALUES(102,2,30);

1 row created.

SQL> INSERT INTO Product_Reorder VALUES(103,3,40);

1 row created.

SQL> INSERT INTO Product_Reorder VALUES(104,4,60);

1 row created.

SQL> INSERT INTO Product_Reorder VALUES(105,5,45);

1 row created.

SQL> select *from Product_Reorder;

PRODUCT_ID WAREHOUSE_ID REORDER_LEVEL
---------- ------------ -------------
       101            1            50
       102            2            30
       103            3            40
       104            4            60
       105            5            45

SQL>
```

```
SQL> CREATE TABLE Employee_Address(
  2      Pincode NUMBER PRIMARY KEY,
  3      City VARCHAR2(50),
  4      State VARCHAR2(50)
  5  );

Table created.

SQL> INSERT INTO Employee_Address VALUES(600080,'Chennai','Tamil Nadu');

1 row created.

SQL> INSERT INTO Employee_Address VALUES(560080,'Bangalore','Karnataka');

1 row created.

SQL> INSERT INTO Employee_Address VALUES(500080,'Hyderabad','Telangana');

1 row created.

SQL> INSERT INTO Employee_Address VALUES(400080,'Mumbai','Maharashtra');

1 row created.

SQL> INSERT INTO Employee_Address VALUES(700080,'Kolkata','West Bengal');

1 row created.

SQL> select *from Employee_Address;

   PINCODE CITY                                               STATE
---------- -------------------------------------------------- --------------------------------------------------
    600080 Chennai                                            Tamil Nadu
    560080 Bangalore                                          Karnataka
    500080 Hyderabad                                          Telangana
    400080 Mumbai                                             Maharashtra
    700080 Kolkata                                            West Bengal

SQL>
```

```
SQL> CREATE TABLE Employee(
  2      Employee_id NUMBER PRIMARY KEY,
  3      Employee_name VARCHAR2(50),
  4      Role VARCHAR2(30),
  5      Phone_number VARCHAR2(10) CHECK(LENGTH(Phone_number)=10),
  6      Pincode NUMBER,
  7      FOREIGN KEY(Pincode) REFERENCES Employee_Address(Pincode)
  8  );

Table created.

SQL> INSERT INTO Employee VALUES
  2  (1,'Anil Kumar','Cashier','9000011211',600080);

1 row created.

SQL> INSERT INTO Employee VALUES
  2  (2,'Ravi Teja','Sales Executive','9080022222',560080);

1 row created.

SQL> INSERT INTO Employee VALUES
  2  (3,'Mahesh Rao','Store Keeper','9100033333',500080);

1 row created.

SQL> INSERT INTO Employee VALUES
  2  (4,'Kiran Patel','Manager','90000544444',400080);

1 row created.

SQL> INSERT INTO Employee VALUES
  2  (5,'Sathish Kumar','Delivery Staff','9000055555',700080);

1 row created.

SQL> select *from Employee;

EMPLOYEE_ID EMPLOYEE_NAME                                      ROLE                           PHONE_NUMB    PINCODE
----------- -------------------------------------------------- ------------------------------ ---------- ----------
          1 Anil Kumar                                         Cashier                        9000011211     600080
          2 Ravi Teja                                          Sales Executive                9080022222     560080
          3 Mahesh Rao                                         Store Keeper                   9100033333     500080
          4 Kiran Patel                                        Manager                        90000544444    400080
          5 Sathish Kumar                                      Delivery Staff                 9000055555     700080

SQL>
```

```
SQL> CREATE TABLE Customer(
  2      Customer_id NUMBER PRIMARY KEY,
  3      Customer_name VARCHAR2(50),
  4      Phone VARCHAR2(10) CHECK(LENGTH(Phone)=10),
  5      Email VARCHAR2(50)
  6  );

Table created.

SQL> INSERT INTO Customer VALUES (1,'Rohit',8881371211,'rohit@gmail.com');

1 row created.

SQL> INSERT INTO Customer VALUES (2,'Meena',8882322122,'meena@gmail.com');

1 row created.

SQL> INSERT INTO Customer VALUES (3,'Kavya',8883993333,'kavya@gmail.com');

1 row created.

SQL> INSERT INTO Customer VALUES (4,'Ajay',88849448844,'ajay@gmail.com');

1 row created.

SQL> INSERT INTO Customer VALUES (5,'Priya',8885556557,'priya@gmail.com');

1 row created.

SQL> select *from Customer;

CUSTOMER_ID CUSTOMER_NAME                                      PHONE      EMAIL
----------- -------------------------------------------------- ---------- --------------------------------------------------
          1 Rohit                                              8881371211 rohit@gmail.com
          2 Meena                                              8882322122 meena@gmail.com
          3 Kavya                                              8883993333 kavya@gmail.com
          4 Ajay                                               88849448844 ajay@gmail.com
          5 Priya                                              8885556557 priya@gmail.com

SQL>
```

```
SQL> CREATE TABLE Purchase_Order_Status(
  2      Status VARCHAR2(20) PRIMARY KEY,
  3      Payment_status VARCHAR2(20)
  4  );

Table created.

SQL> INSERT INTO Purchase_Order_Status VALUES('Ordered','Pending');

1 row created.

SQL> INSERT INTO Purchase_Order_Status VALUES('Shipped','Pending');

1 row created.

SQL> INSERT INTO Purchase_Order_Status VALUES('Received','Paid');

1 row created.

SQL> INSERT INTO Purchase_Order_Status VALUES('Cancelled','Refunded');

1 row created.

SQL> INSERT INTO Purchase_Order_Status VALUES('Processing','Pending');

1 row created.

SQL> select *from Purchase_order_Status;

STATUS               PAYMENT_STATUS
-------------------- --------------------
Ordered              Pending
Shipped              Pending
Received             Paid
Cancelled            Refunded
Processing           Pending

SQL>
```

```
SQL> CREATE TABLE Purchase_Order(
  2      Po_id NUMBER PRIMARY KEY,
  3      Supplier_id NUMBER,
  4      Order_date DATE,
  5      Total_amount NUMBER(10,2),
  6      Status VARCHAR2(20),
  7      Delivery_date DATE,
  8      FOREIGN KEY(Supplier_id) REFERENCES Supplier(Supplier_id),
  9      FOREIGN KEY(Status) REFERENCES Purchase_Order_Status(Status)
 10  );

Table created.

SQL> INSERT INTO Purchase_Order VALUES
  2  (201,1,TO_DATE('10-01-2026','DD-MM-YYYY'),15000,'Ordered',
  3  TO_DATE('15-01-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Purchase_Order VALUES
  2  (202,2,TO_DATE('12-01-2026','DD-MM-YYYY'),22000,'Shipped',
  3  TO_DATE('18-01-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Purchase_Order VALUES
  2  (203,3,TO_DATE('15-01-2026','DD-MM-YYYY'),18000,'Received',
  3  TO_DATE('20-01-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Purchase_Order VALUES
  2  (204,4,TO_DATE('18-01-2026','DD-MM-YYYY'),25000,'Processing',
  3  TO_DATE('25-01-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Purchase_Order VALUES
  2  (205,5,TO_DATE('20-01-2026','DD-MM-YYYY'),12000,'Ordered',
  3  TO_DATE('28-01-2026','DD-MM-YYYY'));

1 row created.

SQL> select *from Purchase_Order;

     PO_ID SUPPLIER_ID ORDER_DAT TOTAL_AMOUNT STATUS              DELIVERY_
---------- ----------- --------- ------------ ------------------- ---------
       201           1 10-JAN-26        15000 Ordered             15-JAN-26
       202           2 12-JAN-26        22000 Shipped             18-JAN-26
       203           3 15-JAN-26        18000 Received            20-JAN-26
       204           4 18-JAN-26        25000 Processing          25-JAN-26
       205           5 20-JAN-26        12000 Ordered             28-JAN-26
```

```
SQL> CREATE TABLE Purchase_Order_Details(
  2      Po_id NUMBER,
  3      Product_id NUMBER,
  4      Batch_no VARCHAR2(20),
  5      Quantity_order NUMBER,
  6      PRIMARY KEY(Po_id,Product_id,Batch_no),
  7      FOREIGN KEY(Po_id) REFERENCES Purchase_Order(Po_id),
  8      FOREIGN KEY(Product_id) REFERENCES Product(Product_id)
  9  );

Table created.

SQL> INSERT INTO Purchase_Order_Details VALUES
  2  (201,101,'B101',50);

1 row created.

SQL> INSERT INTO Purchase_Order_Details VALUES
  2  (202,102,'B102',40);

1 row created.

SQL> INSERT INTO Purchase_Order_Details VALUES
  2  (203,103,'B103',60);

1 row created.

SQL> INSERT INTO Purchase_Order_Details VALUES
  2  (204,104,'B104',80);

1 row created.

SQL> INSERT INTO Purchase_Order_Details VALUES
  2  (205,105,'B105',30);

1 row created.

SQL> select *from Purchase_Order_Details;

     PO_ID PRODUCT_ID BATCH_NO             QUANTITY_ORDER
---------- ---------- -------------------- --------------
       201        101 B101                             50
       202        102 B102                             40
       203        103 B103                             60
       204        104 B104                             80
       205        105 B105                             30

SQL>
```

```
SQL> CREATE TABLE Delivery_Details(
  2      Po_id NUMBER PRIMARY KEY,
  3      Purchase_agent_name VARCHAR2(50),
  4      Shipping_mode VARCHAR2(20),
  5      FOREIGN KEY(Po_id) REFERENCES Purchase_Order(Po_id)
  6  );

Table created.

SQL> INSERT INTO Delivery_Details VALUES(201,'Anil Kumar','Truck');

1 row created.

SQL> INSERT INTO Delivery_Details VALUES(202,'Ravi Teja','Van');

1 row created.

SQL> INSERT INTO Delivery_Details VALUES(203,'Mahesh Rao','Courier');

1 row created.

SQL> INSERT INTO Delivery_Details VALUES(204,'Kiran Patel','Truck');

1 row created.

SQL> INSERT INTO Delivery_Details VALUES(205,'Sathish Kumar','Van');

1 row created.

SQL> select *from Delivery_Details;

     PO_ID PURCHASE_AGENT_NAME                                SHIPPING_MODE
---------- ---------------------------------------------------- --------------------
       201 Anil Kumar                                           Truck
       202 Ravi Teja                                            Van
       203 Mahesh Rao                                           Courier
       204 Kiran Patel                                          Truck
       205 Sathish Kumar                                        Van

SQL>
```

```
SQL> CREATE TABLE Sales_Transaction_Fees(
  2      Payment_type_id NUMBER PRIMARY KEY,
  3      Payment_processing_fee NUMBER(5,2)
  4  );

Table created.

SQL> INSERT INTO Sales_Transaction_Fees VALUES(1,10.5);

1 row created.

SQL> INSERT INTO Sales_Transaction_Fees VALUES(2,15.0);

1 row created.

SQL> INSERT INTO Sales_Transaction_Fees VALUES(3,12.35);

1 row created.

SQL> INSERT INTO Sales_Transaction_Fees VALUES(4,7.35);

1 row created.

SQL> INSERT INTO Sales_Transaction_Fees VALUES(6,10.45);

1 row created.

SQL> select *from Sales_Transaction;
select *from Sales_Transaction
            *
ERROR at line 1:
ORA-00942: table or view does not exist


SQL> select *from Sales_Transaction_Fees;

PAYMENT_TYPE_ID PAYMENT_PROCESSING_FEE
--------------- ----------------------
              1                   10.5
              2                     15
              3                  12.35
              4                   7.35
              6                  10.45

SQL>
```

```
SQL> CREATE TABLE Sales_Transaction_Rates(
  2      Tax_category VARCHAR2(20) PRIMARY KEY,
  3      Tax_rate NUMBER(5,2)
  4  );

Table created.

SQL> INSERT INTO Sales_Transaction_Rates VALUES('GST15',15);

1 row created.

SQL> INSERT INTO Sales_Transaction_Rates VALUES('GST12',12);

1 row created.

SQL> INSERT INTO Sales_Transaction_Rates VALUES('GST18',18);

1 row created.

SQL> INSERT INTO Sales_Transaction_Rates VALUES('GST28',28);

1 row created.

SQL> INSERT INTO Sales_Transaction_Rates VALUES('GST36',36);

1 row created.

SQL> select *from Sales_Transaction_Rates;

TAX_CATEGORY          TAX_RATE
-------------------- ----------
GST15                        15
GST12                        12
GST18                        18
GST28                        28
GST36                        36
```

SQL Plus

```
SQL> CREATE TABLE Sales_Transaction(
  2      Transaction_id NUMBER PRIMARY KEY,
  3      Customer_id NUMBER,
  4      Employee_id NUMBER,
  5      Transaction_date DATE,
  6      Grand_total NUMBER(10,2),
  7      Payment_type_id NUMBER,
  8      FOREIGN KEY(Customer_id) REFERENCES Customer(Customer_id),
  9      FOREIGN KEY(Employee_id) REFERENCES Employee(Employee_id),
 10      FOREIGN KEY(Payment_type_id) REFERENCES Sales_Transaction_Fees(Payment_type_id)
 11  );

Table created.

SQL> INSERT INTO Sales_Transaction VALUES
  2  (301,1,1,TO_DATE('05-02-2026','DD-MM-YYYY'),2500,1);

1 row created.

SQL> INSERT INTO Sales_Transaction VALUES
  2  (302,2,2,TO_DATE('06-02-2026','DD-MM-YYYY'),1800,2);

1 row created.

SQL> INSERT INTO Sales_Transaction VALUES
  2  (303,3,3,TO_DATE('07-02-2026','DD-MM-YYYY'),3200,3);

1 row created.

SQL> INSERT INTO Sales_Transaction VALUES
  2  (304,4,4,TO_DATE('08-02-2026','DD-MM-YYYY'),1500,1);

1 row created.

SQL> INSERT INTO Sales_Transaction VALUES
  2  (305,5,5,TO_DATE('09-02-2026','DD-MM-YYYY'),2100,2);

1 row created.

SQL> select *from Sales_Transaction;

TRANSACTION_ID CUSTOMER_ID EMPLOYEE_ID TRANSACTI GRAND_TOTAL PAYMENT_TYPE_ID
-------------- ----------- ----------- -------- ----------- ---------------
           301           1           1 05-FEB-26        2500               1
           302           2           2 06-FEB-26        1800               2
           303           3           3 07-FEB-26        3200               3
           304           4           4 08-FEB-26        1500               1
           305           5           5 09-FEB-26        2100               2

SQL>
```

```
SQL> CREATE TABLE Sales_Transaction_Details(
  2      Transaction_id NUMBER,
  3      Product_id NUMBER,
  4      Quantity_sold NUMBER,
  5      Unit_price NUMBER(8,2),
  6      Tax_category VARCHAR2(20),
  7      PRIMARY KEY(Transaction_id,Product_id),
  8      FOREIGN KEY(Transaction_id) REFERENCES Sales_Transaction(Transaction_id),
  9      FOREIGN KEY(Product_id) REFERENCES Product(Product_id),
 10      FOREIGN KEY(Tax_category) REFERENCES Sales_Transaction_Rates(Tax_category)
 11  );

Table created.

SQL> INSERT INTO Sales_Transaction_Details VALUES
  2  (301,101,5,50,'GST15');

1 row created.

SQL> INSERT INTO Sales_Transaction_Details VALUES
  2  (302,102,4,60,'GST12');

1 row created.

SQL> INSERT INTO Sales_Transaction_Details VALUES
  2  (303,103,6,70,'GST18');

1 row created.

SQL> INSERT INTO Sales_Transaction_Details VALUES
  2  (304,104,3,80,'GST15');

1 row created.

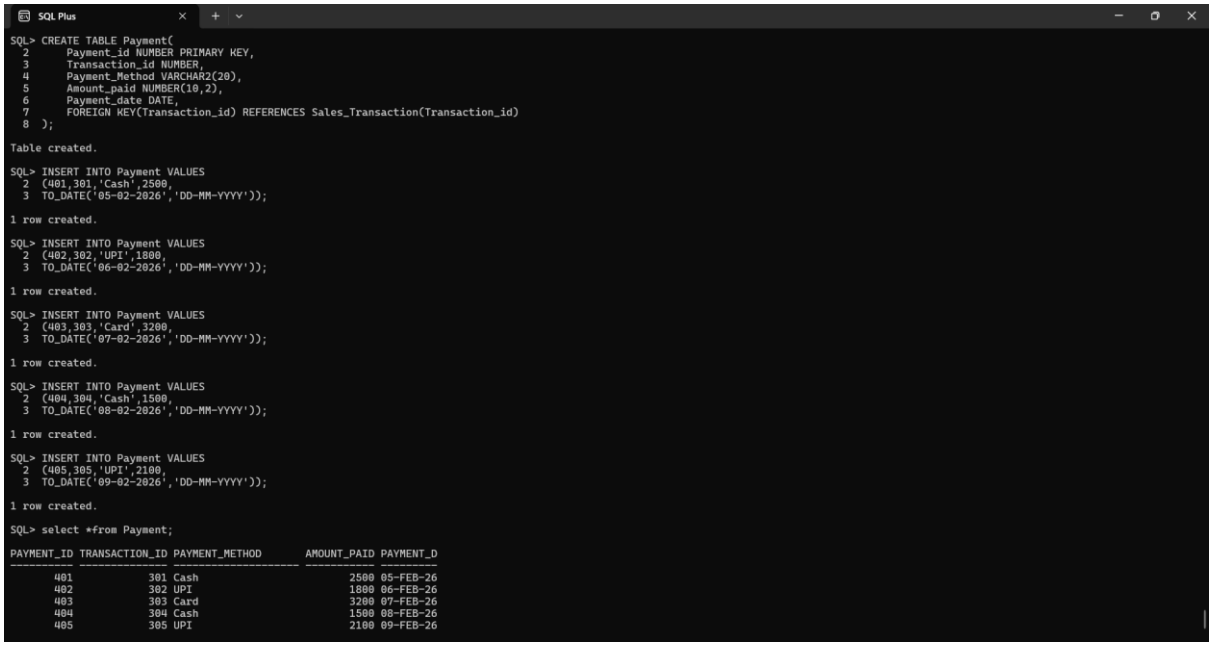
SQL> INSERT INTO Sales_Transaction_Details VALUES
  2  (305,105,7,40,'GST12');

1 row created.
```

```
SQL> select *from Sales_Transaction_Details;

TRANSACTION_ID PRODUCT_ID QUANTITY_SOLD UNIT_PRICE TAX_CATEGORY
-------------- ---------- ------------- ---------- --------------------
           301        101             5         50 GST15
           302        102             4         60 GST12
           303        103             6         70 GST18
           304        104             3         80 GST15
           305        105             7         40 GST12

SQL>
```



```
SQL Plus

SQL> CREATE TABLE Payment(
  2      Payment_id NUMBER PRIMARY KEY,
  3      Transaction_id NUMBER,
  4      Payment_Method VARCHAR2(20),
  5      Amount_paid NUMBER(10,2),
  6      Payment_date DATE,
  7      FOREIGN KEY(Transaction_id) REFERENCES Sales_Transaction(Transaction_id)
  8  );

Table created.

SQL> INSERT INTO Payment VALUES
  2  (401,301,'Cash',2500,
  3  TO_DATE('05-02-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Payment VALUES
  2  (402,302,'UPI',1800,
  3  TO_DATE('06-02-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Payment VALUES
  2  (403,303,'Card',3200,
  3  TO_DATE('07-02-2026','DD-MM-YYYY'));

1 row created.

SQL> INSERT INTO Payment VALUES
  2  (404,304,'Cash',1500,
  3  TO_DATE('08-02-2026','DD-MM-YYYY'));

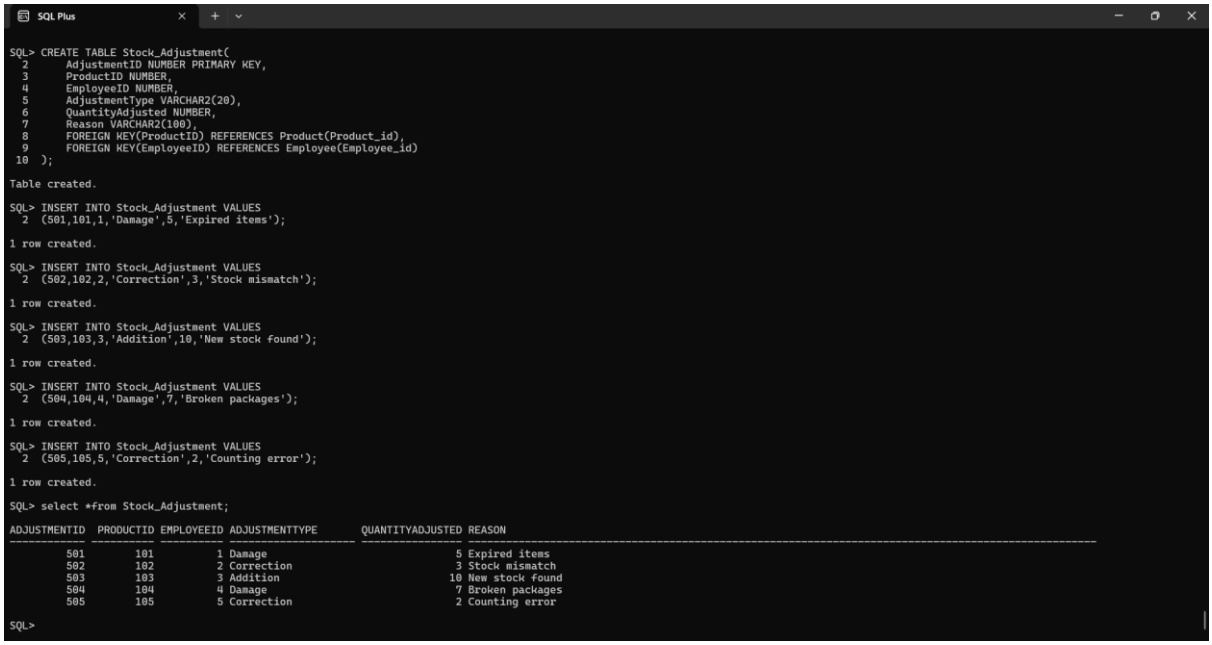
1 row created.

SQL> INSERT INTO Payment VALUES
  2  (405,305,'UPI',2100,
  3  TO_DATE('09-02-2026','DD-MM-YYYY'));

1 row created.

SQL> select *from Payment;

PAYMENT_ID TRANSACTION_ID PAYMENT_METHOD       AMOUNT_PAID PAYMENT_D
---------- -------------- -------------------- ----------- ---------
       401            301 Cash                        2500 05-FEB-26
       402            302 UPI                         1800 06-FEB-26
       403            303 Card                        3200 07-FEB-26
       404            304 Cash                        1500 08-FEB-26
       405            305 UPI                         2100 09-FEB-26
```



```
SQL Plus

SQL> CREATE TABLE Stock_Adjustment(
  2      AdjustmentID NUMBER PRIMARY KEY,
  3      ProductID NUMBER,
  4      EmployeeID NUMBER,
  5      AdjustmentType VARCHAR2(20),
  6      QuantityAdjusted NUMBER,
  7      Reason VARCHAR2(100),
  8      FOREIGN KEY(ProductID) REFERENCES Product(Product_id),
  9      FOREIGN KEY(EmployeeID) REFERENCES Employee(Employee_id)
 10  );

Table created.

SQL> INSERT INTO Stock_Adjustment VALUES
  2  (501,101,1,'Damage',5,'Expired items');

1 row created.

SQL> INSERT INTO Stock_Adjustment VALUES
  2  (502,102,2,'Correction',3,'Stock mismatch');

1 row created.

SQL> INSERT INTO Stock_Adjustment VALUES
  2  (503,103,3,'Addition',10,'New stock found');

1 row created.

SQL> INSERT INTO Stock_Adjustment VALUES
  2  (504,104,4,'Damage',7,'Broken packages');

1 row created.

SQL> INSERT INTO Stock_Adjustment VALUES
  2  (505,105,5,'Correction',2,'Counting error');

1 row created.

SQL> select *from Stock_Adjustment;

ADJUSTMENTID  PRODUCTID EMPLOYEEID ADJUSTMENTTYPE       QUANTITYADJUSTED REASON
------------ ---------- ---------- -------------------- ---------------- --------------------------------------------------------------------------------
         501        101          1 Damage                              5 Expired items
         502        102          2 Correction                          3 Stock mismatch
         503        103          3 Addition                           10 New stock found
         504        104          4 Damage                              7 Broken packages
         505        105          5 Correction                          2 Counting error

SQL>
```

ALL TABLES:

```
SQL> SELECT * FROM Category;

CATEGORY_ID CATEGORY_NAME                                      DESCRIPTION
----------- -------------------------------------------------- ------------------------------------------------------------------------------------------------------------------------------------------------------------
          1 Groceries                                          Daily food items
          2 Beverages                                          Drinks and juices
          3 Personal Care                                      Hygiene products
          4 Snacks                                             Packaged snack items
          5 Household                                          Cleaning products

SQL> SELECT * FROM Product;

PRODUCT_ID PRODUCT_NAME                                         BARCODE UNIT_PRICE CATEGORY_ID SUPPLIER_ID   HSN_CODE
---------- -------------------------------------------------- ---------- ---------- ----------- ----------- ----------
       101 Rice 5kg                                              11111        350           1           1       1001
       102 Apple Juice                                           22222        120           2           2       2001
       103 Shampoo                                               33333        180           3           3       3001
       104 Biscuits                                              44444         40           4           4       1001
       105 Detergent                                             55555        220           5           5       3001

SQL> SELECT * FROM Product_Description;

PRODUCT_ID PRODUCT_DESCRIPTION
---------- ------------------------------------------------------------------------------------------------------------------------------------------------------------
       101 Premium rice quality
       102 1L apple juice bottle
       103 Anti dandruff shampoo
       104 Crunchy biscuits pack
       105 Washing detergent powder

SQL> SELECT * FROM HSN_Tax;

  HSN_CODE   GST_RATE
---------- ----------
      1001          5
      2001         12
      3001         18
      4001         28
      5001          0

SQL>
```

```
SQL> SELECT * FROM Supplier;

SUPPLIER_ID SUPPLIER_NAME                                      PINCODE BANK_ACCOUNT_NUMBER
----------- -------------------------------------------------- ---------- --------------------
          1 Ramesh Kumar                                       600001 123456789012
          2 Suresh Traders                                     560001 234567890123
          3 Anita Enterprises                                  500001 345678901234
          4 Vijay Distributors                                 400001 456789012345
          5 Karthik Wholesale                                  700001 567890123456

SQL> SELECT * FROM Supplier_Address;

   PINCODE CITY                                               STATE
---------- -------------------------------------------------- --------------------------------------------------
    600001 Chennai                                            Tamil Nadu
    560001 Bangalore                                          Karnataka
    500001 Hyderabad                                          Telangana
    400001 Mumbai                                             Maharashtra
    700001 Kolkata                                            West Bengal

SQL> SELECT * FROM Supplier_Bank_Details;

BANK_ACCOUNT_NUMBER  IFSC_CODE           BRANCH_NAME                                        BANK_NAME
-------------------- ------------------- -------------------------------------------------- --------------------------------------------------
123456789012         SBIN000111          Chennai Main Branch                                SBI
234567890123         ICIC000222          Bangalore MG Road Branch                           ICICI
345678901234         HDFC000333          Hyderabad Secunderabad Branch                      HDFC
456789012345         AXIS000444          Mumbai Andheri Branch                              AXIS
567890123456         KKBK000555          Kolkata Park Street Branch                         Kotak

SQL> SELECT * FROM Supplier_Contact;

SUPPLIER_ID      PHONE
----------- ----------
          1 9123456789
          1 9876543210
          2 9012345678
          3 9988776655
          4 9090909090

SQL>
```

```
SQL> SELECT * FROM Warehouse;

WAREHOUSE_ID WAREHOUSE_MANAGER                                 PINCODE
------------ -------------------------------------------------- ----------
           1 Mahesh Varma                                       600070
           2 Deepak Sharma                                      560070
           3 Naveen Reddy                                       500070
           4 Rajesh Patil                                       400070
           5 Subhajit Das                                       700070

SQL> SELECT * FROM Warehouse_Address;

   PINCODE CITY                                               STATE
---------- -------------------------------------------------- --------------------------------------------------
    600070 Chennai                                            Tamil Nadu
    560070 Bangalore                                          Karnataka
    500070 Hyderabad                                          Telangana
    400070 Mumbai                                             Maharashtra
    700070 Kolkata                                            West Bengal

SQL> SELECT * FROM Inventory;

INVENTORY_ID PRODUCT_ID   STOCK_IN  STOCK_OUT CURRENT_STOCK EXPIRY_DA WAREHOUSE_ID
------------ ---------- ---------- ---------- ------------- --------- ------------
           1        101        200         50           142 20-DEC-26            1
           2        102        100         20            75 10-OCT-26            2
           3        103        150         40           118 05-NOV-26            3
           4        104        300        100           193 01-SEP-26            4
           5        105        250         70           169 15-AUG-26            5

SQL> SELECT * FROM Product_Reorder;

PRODUCT_ID WAREHOUSE_ID REORDER_LEVEL
---------- ------------ -------------
       101            1            50
       102            2            30
       103            3            40
       104            4            60
       105            5            45

SQL>
```

```
SQL> SELECT * FROM Purchase_Order_Status;

STATUS               PAYMENT_STATUS
-------------------- --------------------
Ordered              Pending
Shipped              Pending
Received             Paid
Cancelled            Refunded
Processing           Pending

SQL> SELECT * FROM Purchase_Order;

     PO_ID SUPPLIER_ID ORDER_DAT TOTAL_AMOUNT STATUS               DELIVERY_
---------- ----------- --------- ------------ -------------------- ---------
       201           1 10-JAN-26        15000 Ordered              15-JAN-26
       202           2 12-JAN-26        22000 Shipped              18-JAN-26
       203           3 15-JAN-26        18000 Received             20-JAN-26
       204           4 18-JAN-26        25000 Processing           25-JAN-26
       205           5 20-JAN-26        12000 Ordered              28-JAN-26

SQL> SELECT * FROM Purchase_Order_Details;

     PO_ID PRODUCT_ID BATCH_NO             QUANTITY_ORDER
---------- ---------- -------------------- --------------
       201        101 B101                             50
       202        102 B102                             40
       203        103 B103                             60
       204        104 B104                             80
       205        105 B105                             30

SQL> SELECT * FROM Delivery_Details;

     PO_ID PURCHASE_AGENT_NAME                                SHIPPING_MODE
---------- -------------------------------------------------- --------------------
       201 Anil Kumar                                         Truck
       202 Ravi Teja                                          Van
       203 Mahesh Rao                                         Courier
       204 Kiran Patel                                        Truck
       205 Sathish Kumar                                      Van

SQL>
```

```
SQL> SELECT * FROM Employee;

EMPLOYEE_ID EMPLOYEE_NAME                                      ROLE                         PHONE_NUMB    PINCODE
----------- -------------------------------------------------- ---------------------------- ---------- ----------
          1 Anil Kumar                                         Cashier                      9000011211     600080
          2 Ravi Teja                                          Sales Executive              9080022222     560080
          3 Mahesh Rao                                         Store Keeper                 9100033333     500080
          4 Kiran Patel                                        Manager                      9000054444     400080
          5 Sathish Kumar                                      Delivery Staff               9000055555     700080

SQL> SELECT * FROM Employee_Address;

   PINCODE CITY                                               STATE
---------- -------------------------------------------------- --------------------------------------------------
    600080 Chennai                                            Tamil Nadu
    560080 Bangalore                                          Karnataka
    500080 Hyderabad                                          Telangana
    400080 Mumbai                                             Maharashtra
    700080 Kolkata                                            West Bengal

SQL>
```

```
SQL> SELECT * FROM Customer;

CUSTOMER_ID CUSTOMER_NAME                                      PHONE      EMAIL
----------- -------------------------------------------------- ---------- --------------------------------------------------
          1 Rohit                                              8881371211 rohit@gmail.com
          2 Meena                                              8882322122 meena@gmail.com
          3 Kavya                                              8883993333 kavya@gmail.com
          4 Ajay                                               8884944844 ajay@gmail.com
          5 Priya                                              8885556557 priya@gmail.com

SQL>
```

```
SQL> SELECT * FROM Sales_Transaction;

TRANSACTION_ID CUSTOMER_ID EMPLOYEE_ID TRANSACTI GRAND_TOTAL PAYMENT_TYPE_ID
-------------- ----------- ----------- -------- ----------- ---------------
           301           1           1 05-FEB-26        2500               1
           302           2           2 06-FEB-26        1800               2
           303           3           3 07-FEB-26        3200               3
           304           4           4 08-FEB-26        1500               1
           305           5           5 09-FEB-26        2100               2

SQL> SELECT * FROM Sales_Transaction_Details;

TRANSACTION_ID PRODUCT_ID QUANTITY_SOLD UNIT_PRICE TAX_CATEGORY
-------------- ---------- ------------- ---------- --------------------
           301        101             5         50 GST15
           302        102             4         60 GST12
           303        103             6         70 GST18
           304        104             3         80 GST15
           305        105             7         40 GST12

SQL> SELECT * FROM Sales_Transaction_Fees;

PAYMENT_TYPE_ID PAYMENT_PROCESSING_FEE
--------------- ----------------------
              1                   10.5
              2                     15
              3                  12.35
              4                   7.35
              6                  10.45

SQL> SELECT * FROM Sales_Transaction_Rates;

TAX_CATEGORY          TAX_RATE
-------------------- ----------
GST15                        15
GST12                        12
GST18                        18
GST28                        28
GST36                        36

SQL>
```

```
SQL> SELECT * FROM Payment;

PAYMENT_ID TRANSACTION_ID PAYMENT_METHOD       AMOUNT_PAID PAYMENT_D
---------- -------------- -------------------- ----------- ---------
       401            301 Cash                        2500 05-FEB-26
       402            302 UPI                         1800 06-FEB-26
       403            303 Card                        3200 07-FEB-26
       404            304 Cash                        1500 08-FEB-26
       405            305 UPI                         2100 09-FEB-26

SQL> SELECT * FROM Stock_Adjustment;

ADJUSTMENTID  PRODUCTID EMPLOYEEID ADJUSTMENTTYPE       QUANTITYADJUSTED REASON
------------ ---------- ---------- -------------------- ---------------- --------------------------------------------------------------------------------------------------------------------
         501        101          1 Damage                              5 Expired items
         502        102          2 Correction                          3 Stock mismatch
         503        103          3 Addition                           10 New stock found
         504        104          4 Damage                              7 Broken packages
         505        105          5 Correction                          2 Counting error

SQL>
```